

ADR-004 — Schémas MySQL isolés par service (vs bases distinctes)

Champ	Valeur
Statut	Accepté
Date	2026-01-25
Auteurs	Équipe CanBankX

Contexte

Chaque microservice DDD doit posséder ses propres données afin d'éviter le couplage de base de données — antipattern classique des architectures microservices. Trois options ont été évaluées :

1. **Une base MySQL par service** (3 conteneurs MySQL séparés)
2. **Un seul MySQL, un schéma par service** (`db_identity`, `db_account`, `db_payment`)
3. **Base de données partagée, tables partagées** (monolithe de données — à éviter)

Décision

Option 2 : un seul conteneur MySQL 8.4, trois schémas isolés.

Service	Schéma	Tables principales
<code>identity-service</code>	<code>db_identity</code>	<code>clients</code>
<code>account-service</code>	<code>db_account</code>	<code>accounts</code>
<code>payment-service</code>	<code>db_payment</code>	<code>bank_transactions</code> , <code>audit_log</code>

L'utilisateur applicatif `projet` reçoit `GRANT ALL` uniquement sur son schéma. Les JOINS cross-schéma sont architecturalement interdits — les services communiquent via API REST uniquement.

Le script `db-init/init.sql` crée les schémas et applique les grants de façon idempotente à chaque démarrage via le service `db-init`.

Conséquences

Positives

- **Isolation logique** : aucun service ne peut accéder aux données d'un autre (contrainte au niveau des grants MySQL).
- **Opérations simplifiées** : un seul conteneur à monitorer, sauvegarder et maintenir.
- **Faible surcharge** : pas de latence réseau supplémentaire entre services et leur schéma.
- **DDL géré par JPA** : `ddl-auto: update` permet un démarrage rapide en développement sans outil de migration.

Négatives / Risques

- **SPOF base de données** : la panne de l'unique instance MySQL affecte les trois services simultanément. Acceptable pour un contexte académique ; en production, il faudrait MySQL Group Replication ou une base par service.
- **ddl-auto: update non idempotent en production** : pour un environnement de production, il faudrait migrer vers Flyway ou Liquibase afin d'avoir des migrations versionnées et reproductibles.
- **Contention sous forte charge** : les tests de stress (200 VUs, 1 compte partagé) ont révélé une contention sur les verrous de lignes InnoDB dans `bank_transactions`. Le taux de succès des transactions tombe à ~20 % dans ce scénario artificiel (tous les VUs sur le même compte). Avec 20 comptes isolés (1 par groupe de VUs), la contention est répartie et la limite devient la capacité du pool HikariCP (50 connexions). Solution : scaling horizontal de `payment-service` + 2e instance en mode LB.